

Rotate Secrets and certs automation

Wednesday, September 10, 2025 1:03 PM

New registrationEndpointsTroubleshootRefreshDownloadPreview featuresGot feedback?

Starting June 30th, 2020 we will no longer add any new features to Azure Active Directory Authentication Library (ADAL) and Azure Active Directory Graph. We will continue to provide technical support and security updates but we will no longer provide feature updates. Applications will need to be upgraded to Microsoft Authentication Library (MSAL) and Microsoft Graph. [Learn more](#)

All applicationsOwned applicationsDeleted applications

Start typing a display name or application (client) ID to filter these ...Add filters

7 applications found

Display name	Application (client) ID	Created on	Certificates & secrets
woffarcmachines	3e660a5-2759-407a-83d4-e1021b902640	2/24/2025	Expired
woffaautomationsp	ea7c0276-54b8-4579-90ed-2ff649514536	12/5/2023	Current
woffcommsvcspn	f01e9fa8-db0b-48d5-96a0-a8036a9853f	4/22/2025	Current
woffentopenaiap	ce5e1903-54df-47f1-ac71-f51f0eed0c37	7/22/2023	Expired
woffentp0AI	770ed9c0-4a08-4575-9192-cfd5072920b	7/22/2023	Expired
woffentpoisearch	f77c83c9-d03e-4b21-a485-4e995050c81	7/24/2023	Expired
woffentpweb	618b1362-7cc8-4d4e-bae0-5f6e4b8d759a	5/14/2025	Current

```
find_expired_secret_and_rotate_keyvault_stores.ps1
# Get all App Registrations
$appRegistrations = Get-AzADApplication
foreach ($app in $appRegistrations) {
    # Check Password Credentials
    foreach ($cred in $app.PasswordCredentials) {
        $daysRemaining = ($cred.EndDateTime - (Get-Date)).Days
        Write-Host "App: $($app.DisplayName) - Secret expires on:" -NoNewLine
        if ($daysRemaining -le $thresholdDays) {
            Write-Host " $($cred.EndDateTime)" -ForegroundColor Red -BackgroundColor White
            $secretStatus = "Expired"
        } else {
            Write-Host " $($cred.EndDateTime)" -ForegroundColor Green
            $secretStatus = "Active"
        }
    }
}
Write-Host " $($cred.EndDateTime)" -ForegroundColor Green
$secretStatus = "Active"
}
}
App: wolffentp0AI - Secret expires on: 09/11/2025 23:12:57
App: wolffaautomationsp - Secret expires on: 09/11/2025 23:15:09
App: wolffentrandinsp - Secret expires on: 09/11/2025 23:17:27
App: aro-j8k6x4yn - Secret expires on: 09/11/2025 23:19:41
App: wolffentpweb - Secret expires on: 09/11/2025 23:21:54
App: wolffb2cblock - Secret expires on: 09/10/2026 15:34:28
App: arcvcpriimi - Secret expires on: 02/05/2026 16:14:52
App: wolffentpwebsite - Secret expires on: 05/14/2027 23:39:43
App: aro-ton614e4 - Secret expires on: 12/05/2026 20:34:25
App: aro-app-1733345385703 - Secret expires on: 12/04/2026 20:49:47
App: WOLFFCalendarManagerAppspn - Secret expires on: 07/18/2027 00:55:53
App: wolffcommsvcspn - Secret expires on: 04/22/2026 20:38:33
PS C:\WINDOWS\system32>
```

```
Permissions to (Key Vault Managed) Storage :
Tenant ID : e594
Object ID : de79
Application ID :
Display Name :
Permissions to Keys :
Permissions to Secrets : set,
Permissions to Certificates :
Permissions to (Key Vault Managed) Storage :

NetworkAcls : Microsoft.Azure.Commands.KeyVault.Models.PSKeyVaultNetworkRuleSet
NetworkAclsText :
Default Action : Allow
Bypass : Azure
IP Rules : 71.231.245.52/32
Virtual Network Rules :

OriginalVault : Microsoft.Azure.Management.KeyVault.Models.Vault
ResourceId : /subscriptions/5755893a-8056-4ba8-9916-1133c80a80f3/resourceGroups/Adminrg/providers/Microsoft.KeyVault/vaults/wolffkv
VaultName : wolffkv
ResourceGroupName : Adminrg
Location : westus
Tags : {Sequencestart, Team, Purpose, Email...}
TagsTable :
Name Value
-----
Sequencestart 1
Team CSU
Purpose automation and spn use
Email jerrywolff@microsoft.com
businessUnit West OU
Sequencestop 1
owner Jerry wolff

MSI050342
VERBOSE: Performing the operation "Update-AzADApplication.UpdateExpanded" on target
VERBOSE: Performing the operation "Remove-AzADApplicationPassword.RemoveExpanded" on target
Removing old expired secret from App Registration for wolffautomationsp
new secret already created as replacement and stored in wolffkv - wolffautomationsp - Expired
Adding new secret to wolffautomationsp
Tenant/sub : wolffkvsub - 5755893a-8056-4ba8-9916-1133c80a80f3 MSI050342 AzureCloud
wolffkvsub 5755893a-8056-4ba8-9916-1133c80a80f3 MSI050342 AzureCloud

find_expired_secret_and_rotate_and_keyvault_store.ps1 X
# Select subscription
$Subscription = Get-AzSubscription | Out-GridView -Title "Select a subscription:" -PassThru | Select-Object Name, Id
Write-Host "Tenant/sub : $($Subscription.Name) - $($Subscription.TenantId)" -ForegroundColor Green
Set-AzContext -Subscription $($Subscription.Name) -Tenant $($Subscription.TenantId)

$appinfo = get-azadapplication | where displayname -eq $displayname
start-sleep -seconds 60

# Create the service principal
$sp = get-AzADServicePrincipal -ApplicationId $($appinfo.appid)

start-sleep -seconds 60

#####

##### Add my current ip address to the network firewallrule if not there
# Get your current public IP address
$myip = (Invoke-RestMethod -Uri "https://api.ipify.org?format=json").ip
# Get current IP rules
$skv = Get-AzKeyVault -VaultName $vaultName -ResourceGroupName $resourceGroup
$currentips = $skv.NetworkAcls.IpRules.IpAddressOrRange
# Add your IP if it's not already in the list
if ($currentips -notcontains $myip) {
    $updatedips = $currentips + $myip
    # Update the Key Vault network rules
    Update-AzKeyVaultNetworkRuleSet -VaultName $vaultName -ResourceGroupName $resourceGroup -IpAddressRange $updatedips -DefaultAction Deny
} else {
    Write-Host "[X] Added IP $myip to Key Vault firewall rules."
} else {
    Write-Host "[ ] IP $myip is already allowed."
}

$automationip = "20.236.10.163" # for automation MI internal Microsoft IP only not public
$skv = Get-AzKeyVault -VaultName $vaultName -ResourceGroupName $resourceGroup
```

```
Object ID : ar4237c-711c-41d1-b774-bf08f57a328a
Application ID :
Display Name :
Permissions to Keys :
Permissions to Secrets : set, get, list
Permissions to Certificates :
Permissions to (Key Vault Managed) Storage :

NetworkAcls : Microsoft.Azure.Commands.KeyVault.Models.PSKeyVaultNetworkRuleSet
NetworkAclsText :
Default Action : Allow
Bypass : Azure
IP Rules : 71.231.245.52/32
Virtual Network Rules :

OriginalVault : Microsoft.Azure.Management.KeyVault.Models.Vault
ResourceId : /subscriptions/5755893a-8056-4ba8-9916-1133c80a80f3/resourceGroups/Adminrg/providers/Microsoft.KeyVault/vaults/wolffkv
VaultName : wolffkv
ResourceGroupName : Adminrg
Location : westus
Tags : {Sequencestart, Team, Purpose, Email...}
TagsTable :
Name Value
-----
Sequencestart 1
Team CSU
Purpose automation and spn use
Email jerrywolff@microsoft.com
businessUnit West OU
Sequencestop 1
owner Jerry wolff

MSI050342
App: wolffentpOAI - Secret expires on: 09/10/2030 23:30:08
App: wolffautomationsp - Secret expires on: 09/10/2030 23:32:22
App: wolffentraadmins - Secret expires on: 09/10/2030 23:34:41
App: aro-j8k6x4vn - Secret expires on: 09/10/2030 23:36:53
App: wolffentpweb - Secret expires on: 09/10/2030 23:39:08
App: wolffb2cblock - Secret expires on: 09/10/2026 15:34:28
App: arcsvcprnmi - Secret expires on: 02/05/2026 16:14:52
App: wolffentpwebsite - Secret expires on: 05/14/2027 23:39:43
App: aro-ten614e4 - Secret expires on: 12/05/2026 20:34:25
App: aro-app-173345385703 - Secret expires on: 12/04/2026 20:49:47
App: WOLFFCalendarManagerAppspn - Secret expires on: 07/18/2027 00:55:53
App: wolffcommvcsn - Secret expires on: 04/22/2026 20:38:33

find_expired_secret_and_rotate_and_keyvault_store.ps1 X
# Select subscription
$Subscription = Get-AzSubscription | Out-GridView -Title "Select a subscription:" -PassThru | Select-Object Name, Id
Write-Host "Tenant/sub : $($Subscription.Name) - $($Subscription.TenantId)" -ForegroundColor Green
Set-AzContext -Subscription $($Subscription.Name) -Tenant $($Subscription.TenantId)

$appinfo = get-azadapplication | where displayname -eq $displayname
start-sleep -seconds 60

# Create the service principal
$sp = get-AzADServicePrincipal -ApplicationId $($appinfo.appid)

start-sleep -seconds 60

#####

##### Add my current ip address to the network firewallrule if not there
# Get your current public IP address
$myip = (Invoke-RestMethod -Uri "https://api.ipify.org?format=json").ip
# Get current IP rules
$skv = Get-AzKeyVault -VaultName $vaultName -ResourceGroupName $resourceGroup
$currentips = $skv.NetworkAcls.IpRules.IpAddressOrRange
# Add your IP if it's not already in the list
if ($currentips -notcontains $myip) {
    $updatedips = $currentips + $myip
    # Update the Key Vault network rules
    Update-AzKeyVaultNetworkRuleSet -VaultName $vaultName -ResourceGroupName $resourceGroup -IpAddressRange $updatedips -DefaultAction Deny
} else {
    Write-Host "[X] Added IP $myip to Key Vault firewall rules."
} else {
    Write-Host "[ ] IP $myip is already allowed."
}

$automationip = "20.236.10.163" # for automation MI internal Microsoft IP only not public
```

Script

<#
NOTES

THIS CODE-SAMPLE IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY AND/OR FITNESS FOR A PARTICULAR PURPOSE.

This sample is not supported under any Microsoft standard support program or service.

The script is provided AS IS without warranty of any kind. Microsoft further disclaims all implied warranties including, without limitation, any implied warranties of merchantability

or of fitness for a particular purpose. The entire risk arising out of the use or performance

of the sample and documentation remains with you. In no event shall Microsoft, its authors, or anyone else involved in the creation, production, or delivery of the script be liable for any damages whatsoever (including, without limitation, damages for loss of business profits, business interruption, loss of business information, or other pecuniary loss) arising out of the use of or inability to use the sample or documentation, even if Microsoft has been advised of the possibility of such damages, rising out of the use of or inability to use the sample script, even if Microsoft has been advised of the possibility of such damages.

Description: This script automates the rotation of expired secrets for Azure AD App Registrations and securely stores the new secrets in Azure Key Vault. It also ensures that necessary network firewall rules and access policies are updated to allow secret management via managed identity.

☒ Prerequisites
Before running this script, ensure the following:

Modules Installed:

Az module (via Import-Module -Name Az -Force)
Microsoft.Graph.Applications module (via Import-Module Microsoft.Graph.Applications -Verbose)

Managed Identity Access:

The script uses Connect-AzAccount -Identity, so it must be run in a context where a managed identity is available (e.g., Azure Automation, VM with system-assigned identity).

Permissions:

Managed identity must have:

Contributor or Owner access to the subscription.
Key Vault Administrator or equivalent access to manage secrets.
Microsoft Graph API permissions: Application.ReadWrite.All, AppRoleAssignment.ReadWrite.All, Group.Read.All, Directory.Read.All, Directory.ReadWrite.All.

Environment Setup:

Subscription name: "<Subscriptionname>"
Key Vault name: "<keyvaultname>"
Resource group: "<resourcegroup oAdmining>"
App Registration display name: "wolffentpOAI"

 Script Breakdown

1. Module Imports & Authentication
PowerShellImport-Module -Name Az -ForceImport-Module Microsoft.Graph.Applications -VerboseShow more lines

Loads required modules and authenticates using managed identity.

2. Subscription & Graph Connection
PowerShell\$subscription = Get-AzSubscription -SubscriptionName wolffentpsubConnect-MgGraph -Tenant \$(\$subscription.TenantId) -Scopes ...Show more lines

Retrieves subscription context and connects to Microsoft Graph.

3. Function: rotate_expired_secrets

Purpose: Rotates secrets for a given App Registration.
Steps:

Retrieves the App and Service Principal.
Adds current and automation IPs to Key Vault firewall if missing.
Updates Key Vault access policy for the SP.
Creates a new secret and stores it in Key Vault.
Handles errors by retrying and logging IP additions.

4. Function: remove_expired_credential_secrets

Purpose: Deletes expired secrets from App Registrations.
Steps:

Uses Remove-AzADAppCredential to clean up old credentials.

5. Secret Expiration Audit

Purpose: Audits all App Registrations for secrets/certificates expiring within 30 days.
Steps:

Loops through all apps and credentials.
Flags expired items and stores metadata in \$expirationlist.

6. Rotation Execution

Purpose: Executes rotation and cleanup for expired credentials.
Steps:

Loops through \$expirationlist.
Calls remove_expired_credential_secrets and rotate_expired_secrets for each expired item.

 Key Features

Firewall Rule Automation: Dynamically adds IPs to Key Vault network ACLs.
Error Handling: Captures and retries on IP-related access errors.
Credential Lifecycle Management: Ensures secrets are rotated and stored securely.
Logging: Logs IP additions and errors for traceability.

#>
import-module -name az -Force
import-module Microsoft.Graph.Applications -verbose

```

# Connect using managed identity
$context = Connect-AzAccount -Identity

$subscription = get-azsubscription -SubscriptionName wolffentpsub

Connect-MgGraph -tenant $($subscription.TenantId) -Scopes "Application.ReadWrite.All", "AppRoleAssignment.ReadWrite.All", "Group.Read.All", "Directory.Read.All", "Directory.ReadWrite.All"

# Set subscription context
Set-AzContext -Subscription "wolffentpsub"

Function rotate_expired_secrets ([string]$appspn_displayname)
{
    # Define variables
    $vaultname = 'wolffkv'
    $resourceGroup = 'Adminrg'
    $spnname = "$appspn_displayname"
    $displayName = "$appspn_displayname"

    $sp = @()

    # Select subscription
    $subscription = Get-AzSubscription | Out-GridView -Title "Select a subscription:" -PassThru | Select-Object Name, Id, TenantId -First 1

    Write-Host "Tenant/sub : $($subscription.Name) - $($subscription.TenantId)" -ForegroundColor Green

    Set-AzContext -Subscription $($subscription.Name) -Tenant $($subscription.TenantId)

    $appinfo = get-azadapplication | where displayname -eq $displayName

    start-sleep -seconds 60

    # Create the service principal

    $sp = get-AzADServicePrincipal -ApplicationId $($appinfo.appid)

    start-sleep -seconds 60

    #####

    ##### Add my current ip address to the network firewallrule if not there

    # Get your current public IP address
    $myip = (Invoke-RestMethod -Uri "https://api.ipify.org?format=json").ip

    # Get current IP rules
    $kv = Get-AzKeyVault -VaultName $vaultname -ResourceGroupName $resourceGroup
    $currenttpps = $kv.NetworkAcls.IpRules.IpAddressOrRange

    # Add your IP if it's not already in the list
    if ($currenttpps -notcontains $myip) {
        $updatedtpps = $currenttpps + $myip

        # Update the Key Vault network rules
        Update-AzKeyVaultNetworkRuleSet -VaultName $vaultname `
            -ResourceGroupName $resourceGroup `
            -IpAddressRange $updatedtpps `
            -DefaultAction Deny

        Write-Host "[✓] Added IP $myip to Key Vault firewall rules."
    } else {
        Write-Host "[i] IP $myip is already allowed."
    }
}

#$automationip = "20.236.10.163" # for automation MI internal Microsoft IP only not public

$kv = Get-AzKeyVault -VaultName $vaultname -ResourceGroupName $resourceGroup
$currenttpps = $kv.NetworkAcls.IpRules.IpAddressOrRange

# Add the automation IP if not already present
if ($currenttpps -notcontains $automationip) {
    $updatedtpps = $currenttpps + $automationip

    Update-AzKeyVaultNetworkRuleSet -VaultName $vaultname `
        -ResourceGroupName $resourceGroup `
        -IpAddressRange $updatedtpps `
        -DefaultAction Allow

    Write-Host "[✓] Added automation IP $automationip to Key Vault firewall rules."
} else {
    Write-Host "[i] IP $automationip is already allowed."
}

$vault = Get-AzKeyVault -VaultName "$vaultname" -ResourceGroupName "$resourceGroup" -SubscriptionId $($subscription.Id)

Update-AzKeyVaultNetworkRuleSet -VaultName "$vaultname" -Bypass AzureServices

Set-AzKeyVaultAccessPolicy -VaultName "$vaultname" `
    -ObjectId "$($sp.id)" `
    -PermissionsToSecrets set,get,list

$startdate = get-date
$enddate = (Get-Date).AddYears(5)
#used to create expired secrets for testing
# $enddate = (Get-Date).addDays(1)

$customKey = [System.Convert]::ToBase64String([System.Text.Encoding]::UTF8.GetBytes("wolffentpopenaisecret"))

$clientSecret = New-AzADAppCredential -ApplicationId $($sp.AppId) -EndDate $enddate -StartDate $startdate -CustomKeyIdentifier "$customKey"
$secureSecretValue = ConvertTo-SecureString -String $($clientSecret.SecretText) -AsPlainText -Force

Update-AzKeyVaultNetworkRuleSet -DefaultAction Allow -VaultName $vaultname

try {
    Update-AzKeyVault -ResourceGroupName $resourceGroup `
        -VaultName $vaultname `
        -PublicNetworkAccess Enabled

```

```

(Get-AzContext).Account.Id

$UserObjectId = (Get-AzADUser -UserPrincipalName (Get-AzContext).Account.Id).Id

Set-AzKeyVaultAccessPolicy `
    -VaultName "wolffkv" `
    -ObjectId "$UserObjectId" `
    -PermissionsToSecrets get, list, delete, purge, set

#####

Set-AzKeyVaultSecret -VaultName $vaultname -Name $Sp.DisplayName `
    -SecretValue $SecureSecretValue `
    -Tag @(Purpose = "Spautomation"; AppId = "${Sp.id}"; Enddatetime = "${Sp.Enddate}"; keyid = "${Sp.keyid}") `
    -ContentType "${Sp.AppId}"
}
catch {
    $RawError = $Error[0].ToString()

    if ($RawError -match "Client address: (\d{1,3}(\.|\d{1,3}){3})") {
        $ClientIp = $Matches[1]
        Write-Host "Detected client IP: $ClientIp"

        # Add IP to firewall
        Write-Host "Adding $ClientIp to Key Vault firewall..."
        Update-AzKeyVaultNetworkRuleSet -VaultName $vaultName -IpAddressRange $ClientIp
        Write-Host "IP address added. Retrying secret retrieval..."

        # Log the IP and timestamp
        $Timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
        Add-Content -Path $LogFile -Value "$Timestamp - Added IP $ClientIp to $vaultName"

    }
    else {
        Write-Host "Raw error: $RawError"
        return
    }

    Start-Sleep -Seconds 15

    try {

        Set-AzKeyVaultSecret -VaultName $vaultname -Name ${Sp.DisplayName} `
            -SecretValue $SecureSecretValue `
            -Tag @(Purpose = "Spautomation"; AppId = "${Sp.id}"; Enddatetime = "${Sp.Enddate}"; keyid = "${Sp.keyid}") `
            -ContentType "${Sp.AppId}"

    }
    catch {

        Set-AzKeyVaultAccessPolicy `
            -VaultName "wolffkv" `
            -ObjectId "$UserObjectId" `
            -PermissionsToSecrets get, list, delete, purge, set

        ##### To cleanup old secrets if this is re-run with the same app name

        Remove-AzKeyVaultSecret -VaultName $vaultname -Name "${Sp.DisplayName}" -Force -InRemovedState

        Set-AzKeyVaultSecret -VaultName $vaultname -Name ${Sp.DisplayName} `
            -SecretValue $SecureSecretValue `
            -Tag @(Purpose = "Spautomation"; AppId = "${Sp.id}"; Enddatetime = "${Sp.Enddate}"; keyid = "${Sp.keyid}") `
            -ContentType "${Sp.AppId}"

        return
    }
}

}

Function remove_expired_credential_secrets ([string]$appspn_displayname,[string]$keyid)
{

    $expiredsec_tobedeleted = Get-AzADApplication -DisplayName $appspn_displayname | Remove-AzADAppCredential -verbose #-whatif

    Write-host "Removing old expired secret from App Registration for $($appspn_displayname)" -ForegroundColor Red -BackgroundColor white
    Write-host "new secret already created as replacement and stored in $($vaultname) - $($appspn_displayname)" -ForegroundColor black -BackgroundColor white
}

#####
$expirationlist = ""

# Set expiration threshold (e.g., 30 days)
$thresholdDays = 30
$cuttoffDate = (Get-Date).AddDays($thresholdDays)

# Get all App Registrations
$appRegistrations = Get-AzADApplication

foreach ($app in $appRegistrations) {
    # Check Password Credentials
    foreach ($cred in $app.PasswordCredentials) {
        $daysRemaining = ($cred.EndDateTime - (Get-Date)).Days
        Write-Host "App: $($app.DisplayName) - Secret expires on:" -NoNewline

        if ($daysRemaining -le $thresholdDays) {
            Write-Host " $($cred.EndDateTime)" -ForegroundColor Red -BackgroundColor White
            $seccertstatus = "Expired"
        }
        else {
            Write-Host " $($cred.EndDateTime)" -ForegroundColor Green
            $seccertstatus = "Active"
        }
    }

    $seccertobj = new-object PSObject

    $seccertobj | Add-member -MemberType NoteProperty -name App -value $($app.DisplayName)
    $seccertobj | Add-member -MemberType NoteProperty -name Expireson -value $($cred.EndDateTime)
    $seccertobj | Add-member -MemberType NoteProperty -name status -value $seccertstatus
    $seccertobj | Add-member -MemberType NoteProperty -name keyid -value $($cred.KeyId)
    $seccertobj | Add-member -MemberType NoteProperty -name type -value "PasswordCredentials"

```

```

[array]$Expirationlist += $seccertobj
}

# Check Key Credentials (Certificates)
foreach ($cert in $app.KeyCredentials) {
    $certDaysRemaining = ($cert.EndDateTime - (Get-Date)).Days
    Write-Host "App: $($app.DisplayName) - Certificate expires on:" -NoNewline

    if ($certDaysRemaining -le $thresholdDays) {
        Write-Host " $($cert.EndDateTime)" -ForegroundColor Red -BackgroundColor White
    } else {
        Write-Host " $($cert.EndDateTime)" -ForegroundColor Green
    }

    $seccertobj = new-object PSObject

    $seccertobj | Add-member -MemberType NoteProperty -name App -value $($app.DisplayName)
    $seccertobj | Add-member -MemberType NoteProperty -name Expireson -value $($cert.EndDateTime)
    $seccertobj | Add-member -MemberType NoteProperty -name status -value $seccertstatus
    $seccertobj | Add-member -MemberType NoteProperty -name keyid -value $($cert.Keyid)
    $seccertobj | Add-member -MemberType NoteProperty -name type -value "Certkeycredential"

    [array]$Expirationlist += $seccertobj
}
}

foreach ($appspn in $Expirationlist)
{
    if ($appspn.status -eq "Expired" -or $appspn.status -eq $null)
    {
        remove_expired_credential_secrets -appspn_displayname $($appspn.app) -keyid $($appspn.keyid)
        write-host "$($appspn.app) - $($appspn.status)" -ForegroundColor green

        Write-Host "adding new secret/cred to $($appspn.app)" -ForegroundColor blue -BackgroundColor white

        rotate_expired_secrets -appspn_displayname $($appspn.app)
    }
}

# Get all App Registrations
$appRegistrations = Get-AzADApplication

foreach ($app in $appRegistrations) {
    # Check Password Credentials
    foreach ($cred in $app.PasswordCredentials) {
        $daysRemaining = ($cred.EndDateTime - (Get-Date)).Days
        Write-Host "App: $($app.DisplayName) - Secret expires on:" -NoNewline

        if ($daysRemaining -le $thresholdDays) {
            Write-Host " $($cred.EndDateTime)" -ForegroundColor Red -BackgroundColor White
            $seccertstatus = "Expired"
        } else {
            Write-Host " $($cred.EndDateTime)" -ForegroundColor Green
            $seccertstatus = "Active"
        }
    }
}
}

```

